

Veri Setini Train / Test (ve Validation) Olarak Bölme

Veri Neden Bölünür?

- Temel amaç: modelin gerçek dünyada “görmediği / bilmediği” veriler üzerinde ne kadar iyi genelleme (generalize) yapabildiğini test etmektir.
- Eğer model tüm verilerle hem eğitilir hem de test edilirse — model ezberler; bu durumda performans değeri aşırı iyimser çıkar ve gerçek veride başarısı kötü olabilir (overfitting riski).
- Bu yüzden veri ayrılır:
 - **Training set** — modelin öğrendiği, parametrelerini ayarladığı veri,
 - ****Test (ve gerekirse Validation) set** — ****modelin daha önce “görmediği” veriler üzerinde sınındığı veri, kullanılır.**
- Eğer hiperparametre optimizasyonu yapılacaksa genelde üç parçaya ayrılır: train / validation / test. Validation set: model ayarları için; test set: nihai doğrulama için.

Veri Bölme Yöntemleri — Temel Teknikler & Dikkat Edilecek Noktalar Rastgele Bölme (Random Split)

- Veriyi rastgele karıştırıp belirli oranlarda (örneğin %70–%30, %80–%20, %75–%25...) train/test olarak ayırma.
- Basit, hızlı, en yaygın kullanılan yöntem. Ancak: Eğer sınıf dengesizliği varsa ya da önemli alt gruplar (örneğin nadir sınıflar) varsa — bu grupların test ya da train’e denk gelmeme riski var → dengesizlik → yanlış değerlendirme.

Stratified / Dengeli Bölme — Sınıf Dağılımını Koruma

- Özellikle sınıflandırma problemlerinde, sınıfların oransal dağılımını train/test’e benzer tutmak önemli. Bu sayede test seti, tüm sınıf çeşitliliğini temsil eder.
- Eğer sınıf dağılımı dengesiz ise, random split yerine stratified split kullanmak modellerin tarafsızlığı / genellemesi açısından daha sağlıklı.

Zaman Serisi & Sıralı Veri Bölme (Time-Based Split)

- Eğer veri zaman bağımlı (örneğin tarih, zaman serisi, ardışık ölçümler) ise — rastgele karıştırma anlamsız olabilir. Bu durumda veri sıralı bırakılır, önce geçmiş (train), sonra “geleceğe” dair kısmı test olarak ayırmak gerekir.
- Bu tür bölümlenme gerçek dünyaya daha yakın bir test sağlar: “model geçmişten öğrensin, gelecek / yeni gelen veriyi tahmin etsin.”

Üçlü Bölümleme: Train / Validation / Test

- Validation set: model ayarları (hiperparametre seçimi, erken durdurma, model seçimi vb.) için kullanılır. Test set ise yalnızca son değerlendirme için saklanır.

- Eğer validation kullanmıyorsanız — ancak hiperparametre ayarı, model seçimi yaptıysanız — test seti “dolaylı validation” olacağı için test performansı yanıltmış olabilir (çoğu zaman iyimser çıkar). Bu yüzden validation seti veya cross-validation tavsiye edilir.

```
In [1]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Örnek: iris veri seti (veya kendi csv'niz)
from sklearn.datasets import load_iris
data = load_iris(as_frame=True)
df = data.frame
X = df.drop("target", axis=1)
y = df["target"]

# VERİYİ Karıştır + böl: %80 train / %20 test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Train set boyutu:", X_train.shape, "Test set boyutu:", X_test.shape)

# Model eğit
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# Test setinde değerlendirme
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Train set boyutu: (120, 4) Test set boyutu: (30, 4)
Accuracy: 0.9

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	0.82	0.90	0.86	10
2	0.89	0.80	0.84	10
accuracy			0.90	30
macro avg	0.90	0.90	0.90	30
weighted avg	0.90	0.90	0.90	30

Cross-Validation (Çapraz Doğrulama)

Cross-Validation Nedir & Neden Önemli?

- Cross-Validation, bir veri kümesini bir kez değil, birden çok şekilde bölerek — modelin farklı alt-kümelerde tekrarlanan biçimde eğitilip test edildiği bir değerlendirme tekniğidir. Böylece, modelin yalnızca tek bir “train-test split”e bağlı olarak değil, farklı parçalar üzerinde nasıl performans gösterdiği görülür.

- Bu yaklaşımın temel amaçları:
 - Modelin genelleme yeteneğini daha güvenilir / istikrarlı biçimde değerlendirmek.
 - Modelin yalnızca tek bir veri bölmesinde “şanslı” (ya da “şanssız”) olmasının etkisini azaltmak.
 - Overfitting — yani modelin eğitildiği veriyle aşırı uyum yapıp, yeni / görülmemiş veride kötü performans verme riskini saptamak.
- Özellikle veri seti küçükse ya da veri çeşitliliği azsa — Cross-Validation, model değerlendirmesinde daha sağlam & güvenilir sonuç verir.

Özetle: Cross-Validation, modelin “gerçek dünyaya ne kadar genellenebilir?” sorusuna ışık tutar — tek seferlik train / test bölmesinin verdiği yanıltıcı sonuçları azaltır.

Yaygın Cross-Validation Yöntemleri

Yöntem	Açıklama / Ne Zaman Kullanılır	
K-Fold Cross-Validation	Veri kümesi eşit (veya yaklaşık eşit) K alt parçaya bölünür (“fold”). Her iterasyonda bir fold test / validasyon için, kalan K-1 fold eğitim için kullanılır; toplamda K model eğitimi + değerlendirmesi yapılır. Sonuçlar ortalananır.	Genel kullanım. Veri seti yeterince büyük ve bağımsız örnekler varsa.
Stratified K-Fold	Özellikle sınıflandırma problemlerinde — sınıf oranı dengesizse — her fold’da sınıf dağılımının ana veri kümesindeki orana benzer olması sağlanır. Bu sayede “nadirdir / azınlıktadır” gibi sınıflar test setinde ya da train setinde tamamen yer almaz, model dengeli test edilir.	Sınıflandırma + sınıf dengesizliği olan veri setleri.
Leave-One-Out (LOO)	Veri kümesindeki her bir örnek, bir iterasyonda test olarak kullanılır; kalan tüm örnekler ile eğitim yapılır. Bu, K-fold’un özel bir hâlidir (K = n).	Küçük veri setlerinde, maksimum veri kullanımı ile değerlendirme yapılmak istendiğinde. Ancak çok zaman / hesaplama maliyeti olabilir.
Repeated K-Fold	Tek bir K-Fold yeterli gelmediğinde, aynı K-fold bölmesi farklı rastgele bölmelerle birkaç kez tekrarlanır; böylece varyasyonu azaltır.	Veri seti küçük ya da değişken dağılımı yüksekse, performans kestiriminde daha stabil sonuç için.
Diğer varyantlar / özel durumlar (özellikle zaman serisi, gruplanmış veriler vs.)	Örneğin zaman bağımlı veri varsa, rastgele bölme yerine zaman temelli Cross-Validation veya grupta bölme (group-aware) yöntemleri gerekebilir.	Zaman serisi, grouped data, correlated observations vs. durumlarda.

```
In [2]: import numpy as np
from sklearn import datasets
from sklearn.model_selection import KFold, StratifiedKFold, cross_val_score
from sklearn.tree import DecisionTreeClassifier

# Örnek veri: iris
X, y = datasets.load_iris(return_X_y=True)

# Model
clf = DecisionTreeClassifier(random_state=42)
```

```
# 1) Basit K-Fold (örneğin 5 katlı)
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(clf, X, y, cv=kf)
print("K-Fold CV Scores:", scores)
print("Ortalama CV skoru:", scores.mean())

# 2) Stratified K-Fold (sınıf dengesizliği olan veri için)
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores2 = cross_val_score(clf, X, y, cv=skf)
print("Stratified K-Fold CV Scores:", scores2)
print("Ortalama:", scores2.mean())
```

```
K-Fold CV Scores: [1.          0.96666667 0.93333333 0.93333333 0.93333333]
Ortalama CV skoru: 0.9533333333333335
Stratified K-Fold CV Scores: [1.          0.96666667 0.93333333 0.96666667
0.9          ]
Ortalama: 0.9533333333333335
```

Cross-Validation'un Avantajları

- **Daha güvenilir performans tahmini:** Tek bir sabit test setine bağlı kalmadan, veri setinin her kısmı hem eğitim hem test için kullanılmış olur. Bu, performans tahmininin varyansını azaltır,
- **Daha verimli veri kullanımı:** Özellikle veri az ise, tüm veri hem eğitimde hem testte kullanılabilir (farklı fold'larda).
- **Overfitting / underfitting tespiti:** Modelin aşırı ya da az öğrenme eğilimleri fold'lar arası skor değişkenliğinde görülür — bu da modelin stabilitesini analiz etmeyi sağlar.
- **Model / hiperparametre seçimi için güvenilir zemin:** Hangi modelin / parametre konfigürasyonunun daha iyi olduğunu, tek bir rassal split yerine birkaç farklı veri parçacığı üzerinde karşılaştırmak daha objektif.

Cross-Validation'un Dezavantajları

- **Hesaplama maliyeti:** K fold × model eğitimi demek → özellikle veri büyük ya da model karmaşık ise zaman / işlem maliyeti artar.
- **Veri bağımlılığı / sızma (leakage) riski:** Eğer ön işleme (feature scaling, encoding, vb.) tüm veri üzerinden yapıp sonra cross-validation uygulanırsa — bu, "geleceği görmüş gibi" değerlendirme yapar → hatalı sonuçlar çıkar. Bu yüzden preprocessing + modeling aşamalarını pipeline + cross-validation ile birlikte yapmak tavsiye edilir. Bazı araştırmalar, preprocessing'in yanlış sırada yapılmasının Cross-Validation sonuçlarını önyargılı hale getirebileceğini göstermektedir.
- **Stratification / grup bağımlılığı gerekebilir:** Özellikle sınıflar dengesizse ya da veri gruplar hâlindeyse (örneğin aynı kişiden birden fazla ölçüm) — klasik K-Fold yanlış / yanıltıcı olabilir; bu durumda stratified, group-aware veya zaman-serisi özel CV gerekebilir.

In []: